# Homeserver mit Ubuntu, Docker, Pi-hole und Tailscale

Netzwerkweiter Werbeblocker auf eigener Hardware, auch unterwegs nutzbar

**Autor** Tscheky (GitHub: Tscheky0)
**Ausbildung** IT Netzwerktechnik, Schulungszentrum Fohnsdorf
**Datum** Oktober 2026
**Umfang** Hardware, Linux Netzwerkkonfiguration, Container, DNS, VPN und Fehlersuche

Inhalt

Projektüberblick ..... 3
Ziele ..... 3
Netzwerkaufbau ..... 4
Adressübersicht ..... 5
Portübersicht ..... 5
Umsetzung ..... 6
Server per LAN an den Windows PC anbinden ..... 6
Netzwerkkonfiguration mit Netplan ..... 7
Fehlersuche beim Standardgateway ..... 8
Docker und Portainer ..... 8
Pi-hole als Docker Stack ..... 9
Fernzugriff mit Tailscale ..... 11
Blocklisten erweitern ..... 12
SSH Zugang mit Schlüsseln absichern ..... 13
Ergebnis und Kontrolle ..... 14
Probleme und Lösungen im Überblick ..... 15
Sicherheit und Datenschutz ..... 16
Was ich gelernt habe ..... 16

# Projektüberblick

In diesem Projekt habe ich meinen alten Schrott PC zu einem kleinen Homeserver umgebaut. Auf dem Server läuft Ubuntu Server, darauf Docker mit der Verwaltungsoberfläche Portainer und als erster richtiger Dienst ein Pi-hole. Der Pi-hole ist ein DNS Server, der Anfragen an Werbe und Tracking Domains einfach ins Leere laufen lässt. Damit der Werbeblocker nicht nur zuhause, sondern auch am Handy über mobile Daten funktioniert, ist der Server zusätzlich über Tailscale erreichbar.
Die Ausgangslage war nicht ideal. Der Server hat keine WLAN Karte und steht weit weg vom Router. Statt neue Hardware zu kaufen, habe ich ihn per LAN-Kabel direkt an meinen Windows PC gehängt und den Internetzugang über die Windows Internetverbindungsfreigabe weitergegeben. Genau dieser Aufbau hat einige typische Netzwerkprobleme verursacht, die ich in dieser Doku Schritt für Schritt nachvollziehbar mache.

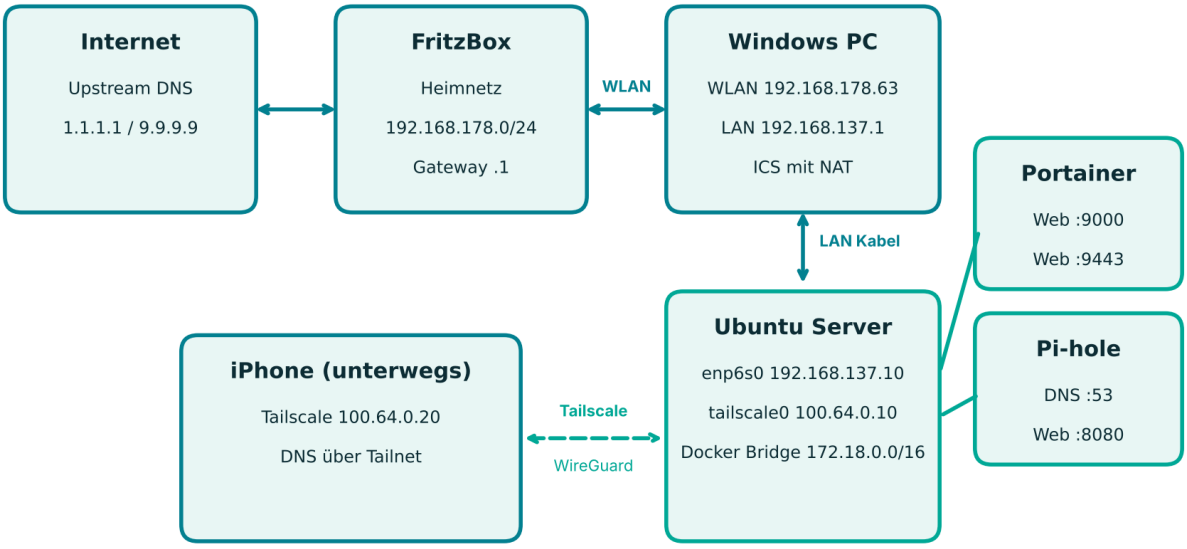
## Ziele

Ich wollte einen eigenen Server betreiben, der rund um die Uhr Dienste bereitstellt und den ich komplett remote per SSH und Weboberfläche verwalten kann. Werbung und Tracker sollten für alle meine Geräte zentral auf DNS Ebene geblockt werden, auch unterwegs. Dabei sollte kein Port ins Internet geöffnet werden, weil mein Anschluss ohnehin hinter CGNAT liegt und offene Ports ein unnötiges Risiko wären.

| Komponente | Aufgabe im Projekt |
|---|---|
| Alter PC, 4 Kerne, 4 GB RAM | Hardware für den Homeserver |
| Ubuntu Server LTS | Betriebssystem ohne Desktop, verwaltet per SSH |
| Netplan | Netzwerkkonfiguration des Servers (IP, Gateway, DNS) |
| Windows ICS | Internetverbindungsfreigabe vom WLAN auf den LAN Port, inklusive DHCP und NAT |
| OpenSSH | Fernzugriff auf die Kommandozeile vom Windows PC |
| Docker | Container Laufzeitumgebung für alle Dienste |
| Portainer CE | Weboberfläche zur Verwaltung von Containern und Stacks |
| Pi-hole | DNS Server mit Blocklisten als netzwerkweiter Werbeblocker |
| Tailscale | Mesh VPN auf WireGuard Basis, Zugriff von unterwegs ohne Portfreigabe |
| Hagezi Pro und StevenBlack | Blocklisten mit zusammen rund 280.000 Domains |

# Netzwerkaufbau

Das Bild zeigt den fertigen Aufbau. Der Windows PC hängt per WLAN an der FritzBox und bildet über seinen LAN Port ein eigenes kleines Netz für den Server. Windows übernimmt dabei automatisch die Rolle als Router mit NAT und vergibt Adressen aus dem Bereich 192.168.137.0/24. Das Handy kommt nicht über das Heimnetz, sondern über das Tailscale Netz (100.64.0.0/10) zum Server.






Windows PC



Ubuntu Server

## Adressübersicht

| Gerät oder Schnittstelle | Adresse | Bemerkung |
|---|---|---|
| FritzBox | 192.168.178.1 | Standardgateway im Heimnetz |
| Windows PC, WLAN | 192.168.178.63 | per DHCP von der FritzBox |
| Windows PC, Ethernet | 192.168.137.1 | Gateway und DNS für den Server (ICS) |
| Ubuntu Server, enp6s0 | 192.168.137.10/24 | statisch über Netplan |
| Ubuntu Server, tailscale0 | 100.64.0.10 | Tailscale Adresse, DNS Server für das Tailnet |
| Docker Bridge des Stacks | 172.18.0.0/16 | internes Netz des Pi-hole Containers |
| iPhone, Tailscale | 100.64.0.20 | nutzt den Pi-hole als DNS |

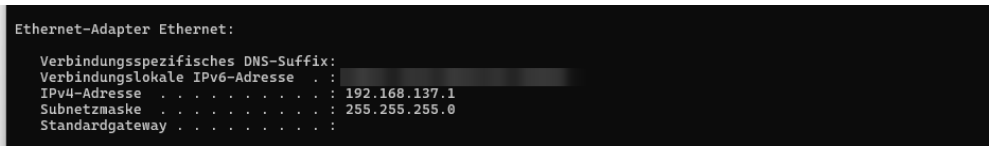
## Portübersicht

| Dienst | Port | Erreichbar über |
|---|---|---|
| SSH | 22/tcp | LAN zum Windows PC |
| Pi-hole DNS | 53/tcp und 53/udp | nur 192.168.137.10 und 100.64.0.10 |
| Pi-hole Weboberfläche | 8080/tcp | LAN zum Windows PC |
| Portainer HTTP | 9000/tcp | LAN zum Windows PC |
| Portainer HTTPS | 9443/tcp | LAN zum Windows PC |

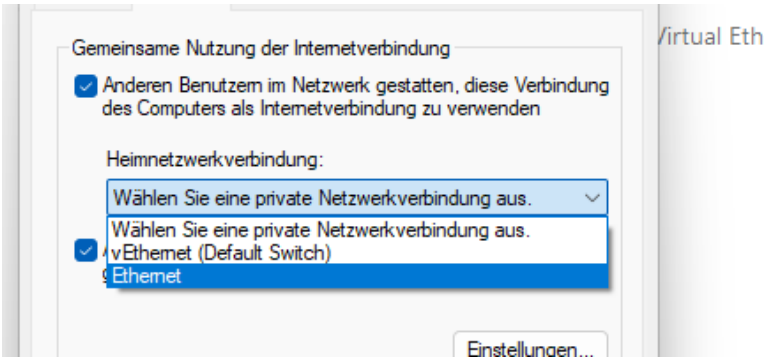
# Umsetzung

## Server per LAN an den Windows PC anbinden

Weil der Server weder WLAN noch eine Kabelverbindung zum Router hat, habe ich ihn direkt mit dem LAN Port meines Windows PCs verbunden. Mit ipconfig habe ich zuerst geprüft, welche Adapter es gibt. Der WLAN Adapter hängt im Heimnetz 192.168.178.0/24, der Ethernet Adapter hat nach Aktivierung der Freigabe automatisch die Adresse 192.168.137.1 bekommen.



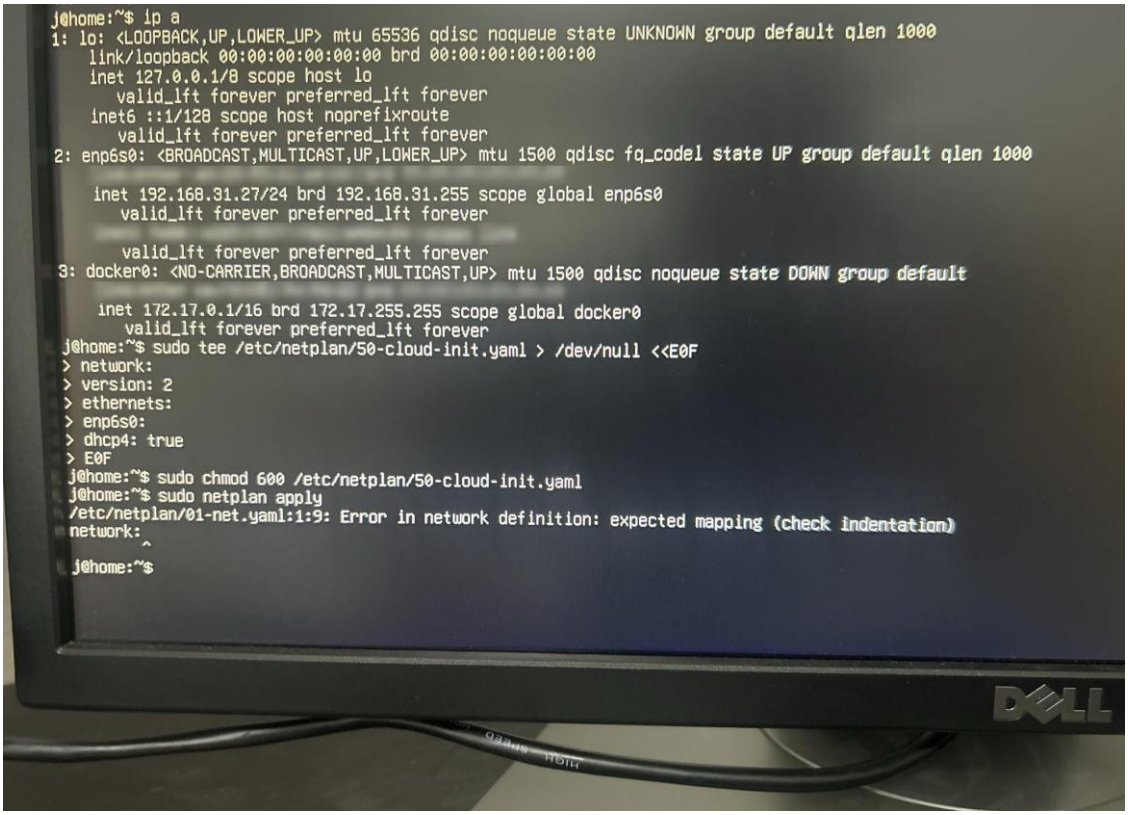

Die Internetverbindungsfreigabe wird in den Eigenschaften des WLAN Adapters aktiviert (ncpa.cpl, Reiter Freigabe). Wichtig ist, dass darunter auch das Zielnetz ausgewählt wird, in meinem Fall Ethernet. Genau das hatte ich zuerst vergessen. Die Freigabe war zwar angehakt, aber ohne ausgewählten Adapter gab es auf dem Kabel weder DHCP noch Routing.



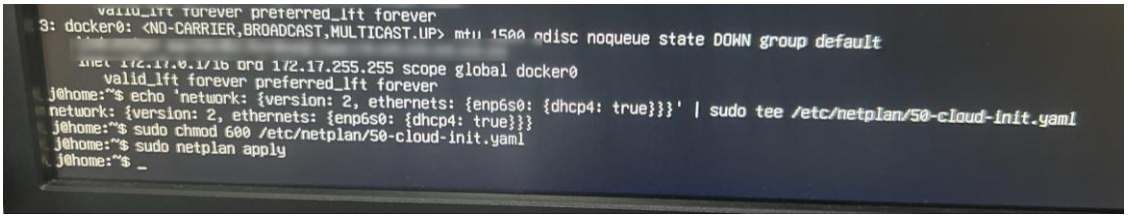
## Netzwerkkonfiguration mit Netplan

Der nächste Stolperstein war der Server selbst. Mit ip a zeigte sich, dass der Adapter enp6s0 noch eine feste Adresse 192.168.31.27/24 aus der Installation hatte. Mit diesem Subnetz konnte er den Windows PC natürlich nicht erreichen, weder Ping noch SSH funktionierten.

Beim Umstellen auf DHCP meldete netplan apply dann einen YAML Fehler. Die Ursache war eine zweite, alte Konfigurationsdatei 01-net.yaml mit kaputter Einrückung. Netplan liest alle Dateien im Ordner /etc/netplan ein, deshalb blockierte diese Datei jede Änderung.



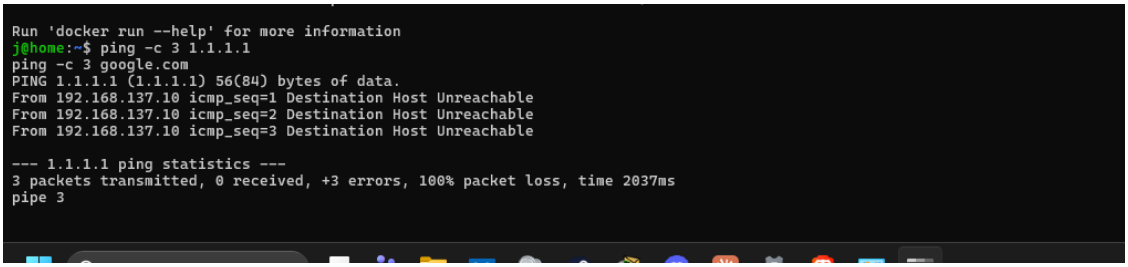

Die Lösung war, die alte Datei wegzuschieben und die neue Konfiguration in YAML Flow Syntax in eine einzige Zeile zu schreiben. So können beim Abtippen am Server keine Einrückungsfehler mehr passieren. Bei meinem ersten Versuch hatte ich noch den falschen Adapternamen enp3s0 verwendet, was man im nächsten Foto sieht. Mit enp6s0 lief netplan apply dann fehlerfrei durch.



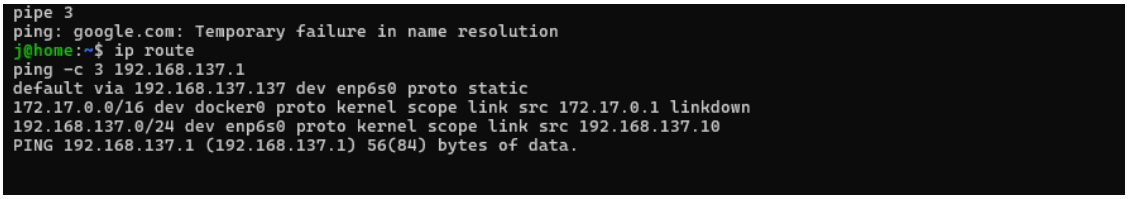

```
sudo mv /etc/netplan/01-net.yaml ~/01-net.yaml.bak
echo 'network: {version: 2, ethernets: {enp6s0: {dhcp4: true}}}'
| sudo tee /etc/netplan/50-cloud-init.yaml
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
sudo netplan apply
```

## Fehlersuche beim Standardgateway

Damit sich die Serveradresse nicht ändert, habe ich danach auf eine statische IP umgestellt. SSH funktionierte sofort, aber der Server kam nicht ins Internet. Der Ping auf 1.1.1.1 meldete Destination Host Unreachable, und zwar vom Server selbst. Das bedeutet, dass der Server nicht einmal sein Gateway findet.

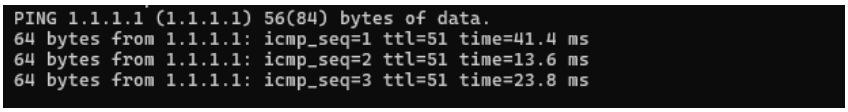



Ein Blick in die Routingtabelle mit ip route zeigte den Fehler sofort: Als Standardgateway war 192.168.137.137 eingetragen statt 192.168.137.1. Ein klassischer Tippfehler, der aber zeigt, warum man bei Netzwerkproblemen systematisch von unten nach oben prüft: zuerst die eigene IP, dann das Gateway, dann eine öffentliche IP und erst zum Schluss die Namensauflösung.




```
echo 'network: {version: 2, ethernets: {enp6s0: {addresses:
[192.168.137.10/24], routes: [{to: default, via: 192.168.137.1}],
nameservers: {addresses: [192.168.137.1, 1.1.1.1]}}}}' | sudo tee
/etc/netplan/50-cloud-init.yaml
sudo netplan apply
ip route
ping -c 3 google.com
```

Nach der Korrektur antwortete 1.1.1.1 zuverlässig, und auch die Namensauflösung funktionierte.

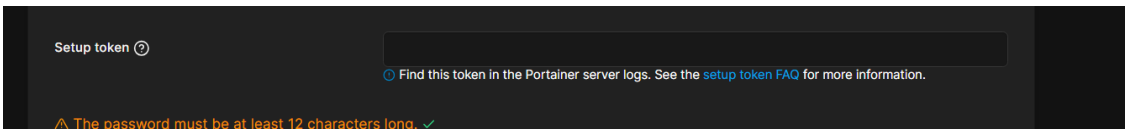



## Docker und Portainer

Docker habe ich aus den Ubuntu Paketquellen installiert. Für die Verwaltung nutze ich Portainer, damit ich Container und Stacks bequem im Browser vom Windows PC aus anlegen, starten und überwachen kann. Portainer läuft selbst als Container und bekommt Zugriff auf den Docker Socket.

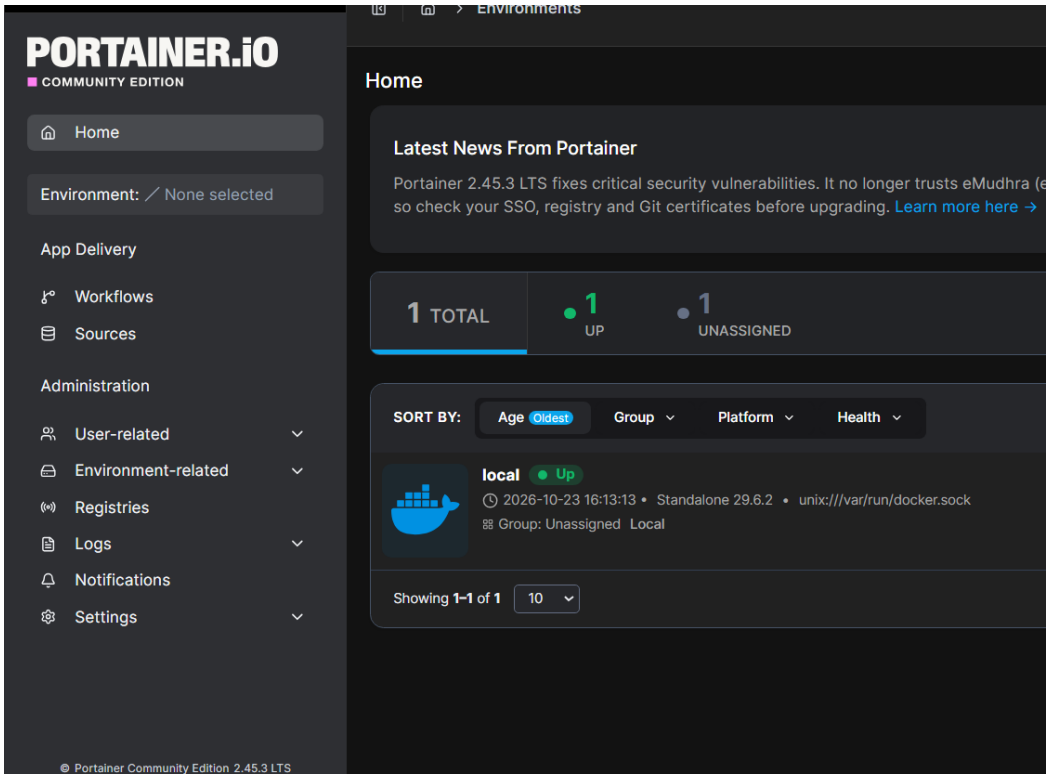
```
sudo apt install docker.io
sudo docker run -d -p 9443:9443 -p 9000:9000 --name portainer --
restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data portainer/portainer-ce
```

Neuere Portainer Versionen verlangen beim ersten Start einen Setup Token, der nur in den Container Logs steht. Damit wird verhindert, dass jemand anderes im Netz schneller den Admin Account anlegt.




```
sudo docker logs portainer 2>&1 | grep -i token
```

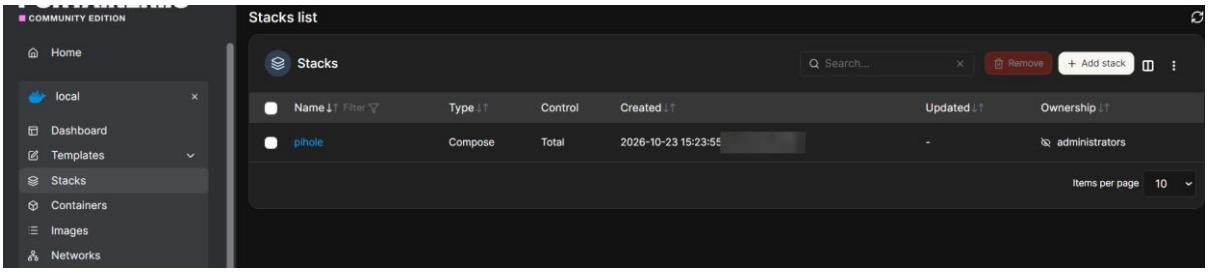
Edge Compute habe ich übersprungen, weil der Docker Host lokal läuft. Danach erscheint das Environment local mit allen Ressourcen des Servers.




Zusätzlich zu HTTPS auf Port 9443 habe ich Port 9000 für HTTP freigegeben, weil das selbst signierte Zertifikat in manchen Browsern Probleme gemacht hat. Beide Ports sind nur im Netz zwischen Server und Windows PC erreichbar.

## Pi-hole als Docker Stack

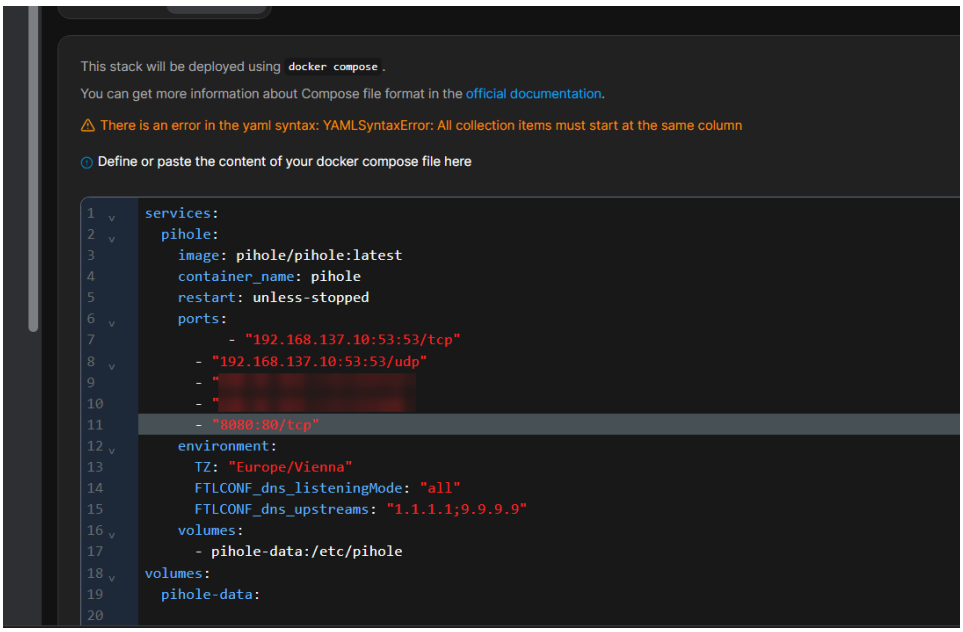
Den Pi-hole habe ich als Stack in Portainer angelegt, also als Docker Compose Datei. Der Vorteil ist, dass die komplette Konfiguration in einer Datei steht und jederzeit nachvollziehbar und wiederherstellbar ist.



Besonders wichtig sind die Portbindungen. Port 53 ist nicht auf allen Schnittstellen offen, sondern gezielt nur auf der LAN Adresse und der Tailscale Adresse des Servers. So beantwortet der Pi-hole keine Anfragen aus fremden Netzen. Die Weboberfläche liegt auf Port 8080, damit Port 80 frei bleibt. Der Eintrag dns im Container sorgt dafür, dass der Pi-hole selbst Blocklisten herunterladen kann, auch wenn er gerade erst startet.

```
services:
  pihole:
    image: pihole/pihole:latest
    container_name: pihole
    restart: unless-stopped
    dns:
      - 127.0.0.1
      - 1.1.1.1
    ports:
      - "192.168.137.10:53:53/tcp"
      - "192.168.137.10:53:53/udp"
      - "100.64.0.10:53:53/tcp"
      - "100.64.0.10:53:53/udp"
      - "8080:80/tcp"
    environment:
      TZ: "Europe/Vienna"
      FTLCONF_dns_listeningMode: "all"
      FTLCONF_dns_upstreams: "1.1.1.1;9.9.9.9"
    volumes:
      - pihole-data:/etc/pihole
volumes:
  pihole-data:
```

Auch hier war YAML die größte Fehlerquelle. Schon ein einziges Leerzeichen zu viel führt zum Fehler, dass alle Listeneinträge in derselben Spalte beginnen müssen. Ich habe daraus gelernt, Compose Dateien immer komplett zu ersetzen statt einzelne Zeilen von Hand einzurücken.



Das Admin Passwort für die Weboberfläche wird direkt im Container gesetzt:

```
sudo docker exec -it pihole pihole setpassword
```

## Fernzugriff mit Tailscale

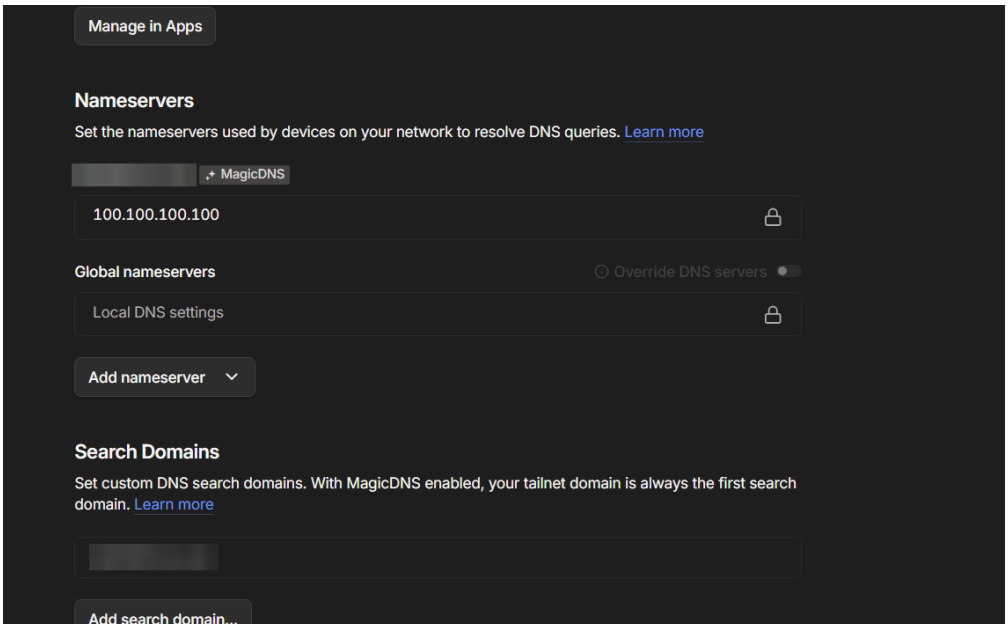
Mein Handy hängt im Heimnetz der FritzBox und unterwegs im Mobilfunknetz, kommt also nicht direkt an den Server hinter dem Windows PC. Eine Portfreigabe kam nicht in Frage, weil mein Internetanschluss hinter CGNAT liegt und ein offener DNS Port im Internet ein Sicherheitsrisiko wäre. Tailscale löst beides. Es baut ein verschlüsseltes WireGuard Netz zwischen den Geräten auf und kommt dabei ohne eingehende Portfreigaben aus.

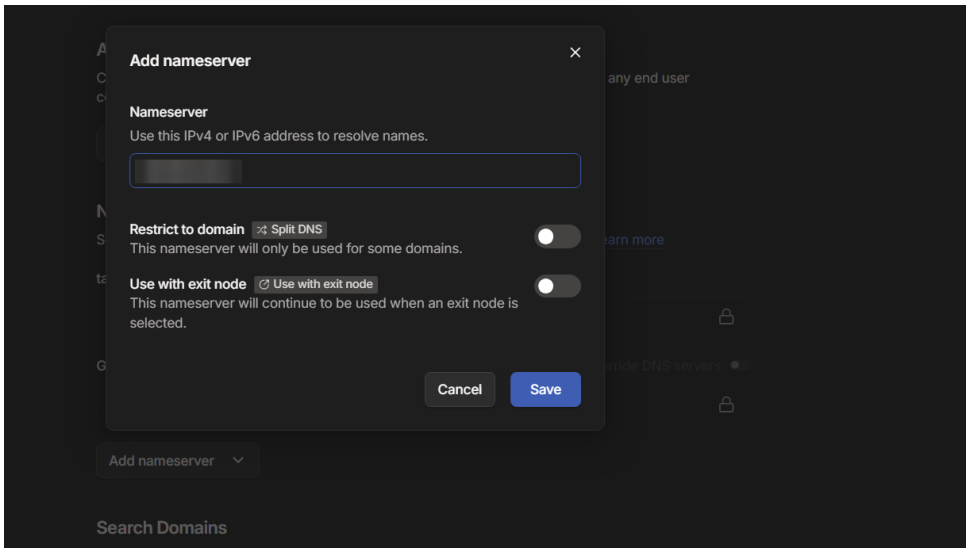
```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
tailscale ip -4
```

Damit Docker Port 53 auf der Tailscale Adresse binden kann, musste ich den lokalen DNS Stub von systemd-resolved abschalten, der standardmäßig auf Port 53 lauscht.

```
sudo sed -i 's/#DNSStubListener=yes/DNSStubListener=no/'
/etc/systemd/resolved.conf
sudo ln -sf /run/systemd/resolve/resolv.conf /etc/resolv.conf
sudo systemctl restart systemd-resolved
```

In der Tailscale Admin Konsole habe ich anschließend die Tailscale Adresse des Servers als globalen Nameserver eingetragen und Override DNS servers aktiviert. Ab dann fragt jedes Gerät im Tailnet automatisch den Pi-hole.

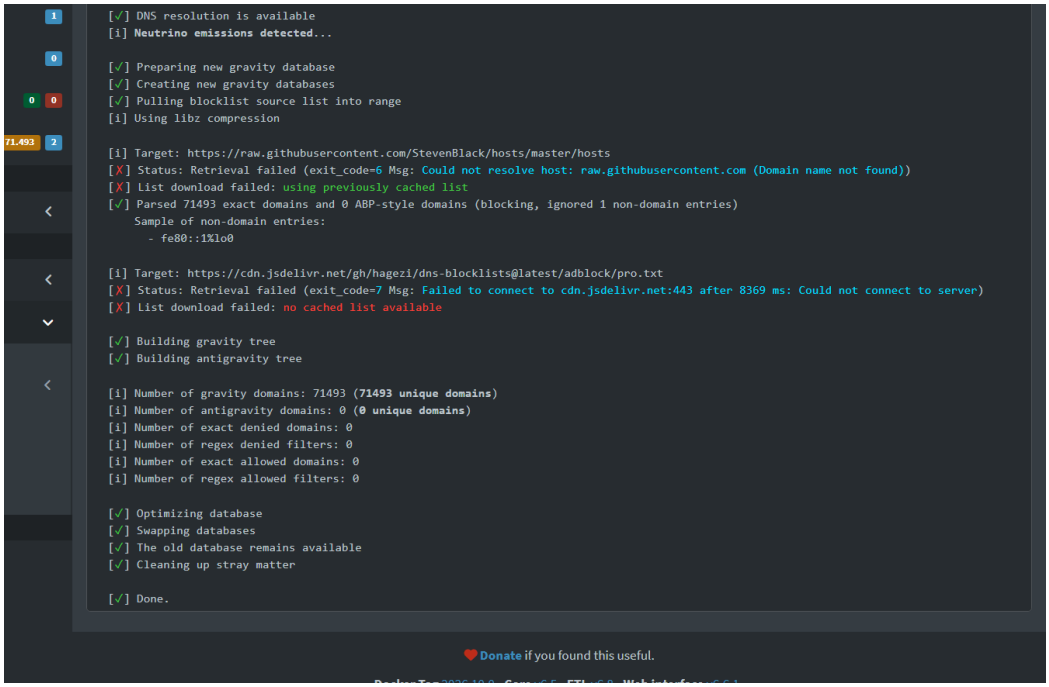





Die Kontrolle am Server mit sudo docker ps zeigt beide Container im Zustand healthy. In der Spalte Ports sieht man, dass Port 53 tatsächlich nur auf 192.168.137.10 und 100.64.0.10 gebunden ist.

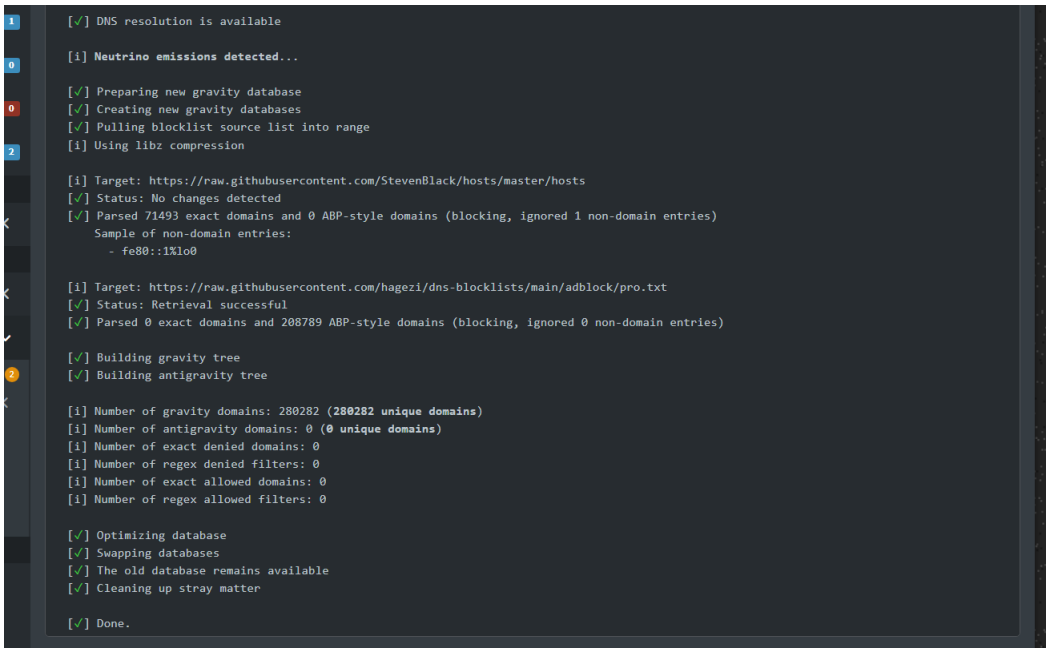
## Blocklisten erweitern

Mit der Standardliste von StevenBlack blockt der Pi-hole rund 71.000 Domains. Das war mir zu wenig, deshalb habe ich die Hagezi Pro Liste ergänzt. Beim ersten Update der Listen schlug der Download allerdings fehl. Der Container konnte selbst keine Namen auflösen, weil er beim Start seinen eigenen DNS Dienst noch nicht bereit hatte, und das CDN jsDelivr war nicht erreichbar.




Gelöst habe ich das mit zwei Änderungen. Der Container bekommt mit dem Eintrag dns im Stack zusätzlich 1.1.1.1 als Fallback, und die Liste wird direkt von GitHub statt über das CDN geladen. Danach liefen beide Downloads durch, und die Zahl der geblockten Domains stieg auf 280.282.

```
https://raw.githubusercontent.com/hagezi/dns-
blocklists/main/adblock/pro.txt
```




## SSH Zugang mit Schlüsseln absichern

Zum Abschluss habe ich den SSH Zugang gehärtet. Bisher konnte man sich mit Benutzername und Passwort anmelden, was anfällig für Brute Force Angriffe ist. Stattdessen nutze ich jetzt ein Schlüsselpaar nach dem Ed25519 Verfahren. Der private Schlüssel bleibt auf meinem Windows PC und ist zusätzlich mit einer Passphrase geschützt, auf dem Server liegt nur der öffentliche Schlüssel.
Den Schlüssel habe ich in PowerShell erzeugt und danach auf den Server übertragen. Weil Windows kein ssh-copy-id mitbringt, wird der öffentliche Schlüssel per Pipe direkt in die Datei authorized_keys geschrieben, inklusive der richtigen Dateirechte.

```
ssh-keygen -t ed25519
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh j@192.168.137.10
"mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >>
~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Erst nachdem die Anmeldung mit Schlüssel in einem zweiten Fenster funktioniert hat, habe ich den Passwort Login abgeschaltet. Die bestehende Sitzung blieb dabei offen, damit ich mich bei einem Fehler nicht aussperre. Die Einstellungen stehen in einer eigenen Datei mit dem Präfix 00, weil sshd bei doppelten Optionen den zuerst gelesenen Wert verwendet. So hat meine Datei Vorrang vor der Ubuntu Standardkonfiguration. Mit sshd -t prüfe ich die Konfiguration vor dem Neustart auf Fehler.

```
echo -e "PasswordAuthentication no\nKbdInteractiveAuthentication
no\nPermitRootLogin no" | sudo tee /etc/ssh/sshd_config.d/00-
hardening.conf
sudo sshd -t && sudo systemctl restart ssh
```

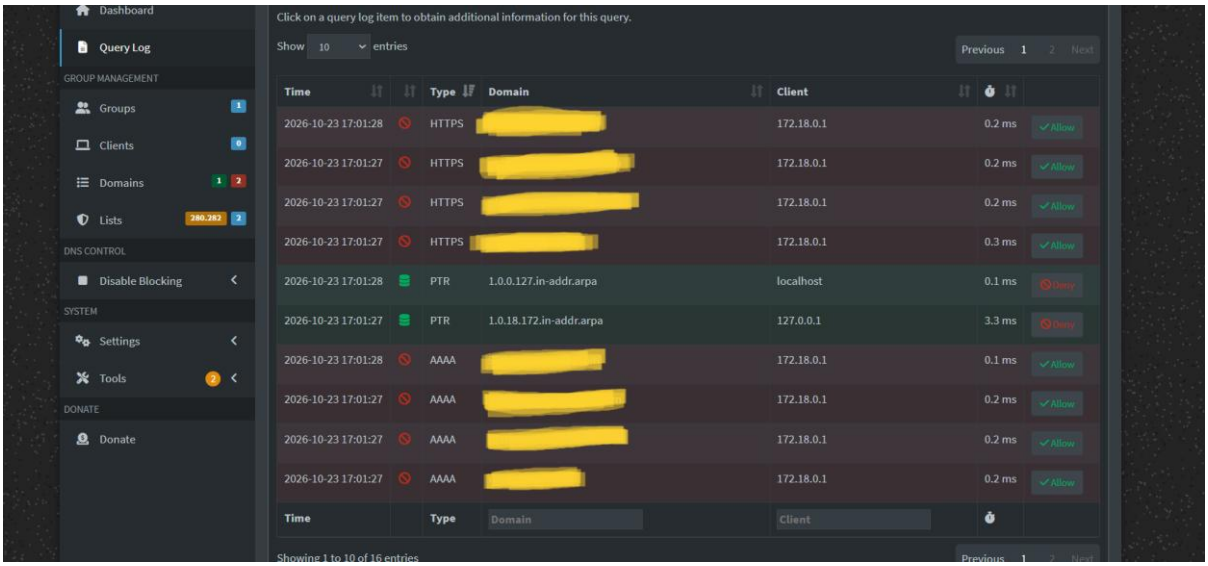
Seitdem kommt nur noch rein, wer meinen privaten Schlüssel hat. Passwort Login und Root Login sind komplett deaktiviert. Wichtig ist eine Sicherung des privaten Schlüssels an einem sicheren Ort, denn ohne ihn ist der Zugang nur noch direkt am Server möglich.

# Ergebnis und Kontrolle

In der Clientübersicht des Pi-hole sieht man, welche Geräte ihn tatsächlich nutzen. Der Windows PC erscheint als 192.168.137.1. Alle Anfragen aus dem Tailnet erscheinen gesammelt als 172.18.0.1, das ist das Gateway der Docker Bridge. Docker übersetzt die Quelladresse beim Weiterleiten in den Container, deshalb sieht der Pi-hole hier nicht die echte Tailscale Adresse des Handys.




Im Query Log sieht man, dass die Werbe und Tracking Anfragen vom Handy zuverlässig geblockt werden. Domains, MAC Adressen, Tailscale Kennungen und Benutzernamen sind in dieser Doku unkenntlich gemacht oder durch Beispieladressen ersetzt.




Damit ist das Ziel erreicht. Der Server läuft dauerhaft, ist vollständig per SSH und Browser verwaltbar, und meine Geräte sind zuhause wie unterwegs durch einen eigenen DNS Filter mit über 280.000 Einträgen geschützt, ohne dass ein einziger Port ins Internet offen ist.

# Probleme und Lösungen im Überblick

| Problem | Ursache | Lösung |
|---|---|---|
| Server per Ping und SSH nicht erreichbar | Alte statische IP 192.168.31.27 aus der Installation, falsches Subnetz | Netplan auf DHCP und später auf 192.168.137.10/24 umgestellt |
| netplan apply meldet expected mapping | Zweite Datei 01-net.yaml mit fehlerhafter Einrückung | Datei weggeschoben, Konfiguration in YAML Flow Syntax geschrieben |
| Keine Adresse per DHCP | Bei der Windows Freigabe war kein Heimnetzwerkadapter ausgewählt | Ethernet als Ziel der Freigabe ausgewählt |
| Destination Host Unreachable | Gateway als 192.168.137.137 statt 192.168.137.1 eingetragen | Mit ip route gefunden und in Netplan korrigiert |
| Portainer verlangt Setup Token | Schutz vor fremdem Admin Account | Token aus docker logs ausgelesen |
| YAML Fehler im Stack Editor | Uneinheitliche Einrückung der Listeneinträge | Compose Datei komplett ersetzt |
| Port 53 nicht auf Tailscale bindbar | systemd-resolved belegt Port 53 | DNSStubListener deaktiviert |
| Blocklisten laden nicht | Container ohne funktionierende Namensauflösung, CDN nicht erreichbar | Fallback DNS im Container und Liste direkt von GitHub |
| Handy zeitweise ohne Filter | Akkuoptimierung beendet die VPN App | Dauerhaft aktives VPN und Akkusparen für die App deaktiviert |

# Sicherheit und Datenschutz

Ein DNS Server sieht sehr genau, welche Dienste ein Gerät nutzt. Deshalb habe ich bei der Umsetzung bewusst auf ein paar Punkte geachtet.

**Keine offenen Ports ins Internet.** Der Zugriff von außen läuft ausschließlich über Tailscale. Ein öffentlich erreichbarer DNS Server könnte sonst für DNS Amplification Angriffe missbraucht werden.

**Gezielte Portbindung.** Port 53 ist nur auf der LAN und der Tailscale Adresse gebunden, nicht auf 0.0.0.0.

**Geräteverwaltung im Tailnet.** Mit tailscale status kontrolliere ich regelmäßig, welche Geräte im Tailnet angemeldet sind. Für den Dauerbetrieb aktiviere ich Device approval, damit jedes neue Gerät erst von mir freigegeben werden muss, und sichere das Konto mit Zwei Faktor Anmeldung ab.

**Protokollierung.** Pi-hole speichert, welches Gerät welche Domain abgefragt hat. Ich werte nur meine eigenen Geräte aus. Wer andere Geräte über den Pi-hole laufen lässt, sollte die Personen informieren oder unter Settings und Privacy den anonymen Modus nutzen, das ist auch aus Sicht der DSGVO der saubere Weg.

**Zugangsdaten.** Portainer und Pi-hole haben eigene, lange Passwörter. SSH läuft nur noch mit Ed25519 Schlüssel, Passwort Login und Root Login sind deaktiviert.

# Was ich gelernt habe

Am meisten hat mir die Fehlersuche gebracht. Fast jedes Problem in diesem Projekt war ein Netzwerkproblem auf Schicht 2 oder 3, und jedes Mal hat sich der gleiche Ablauf bewährt: eigene Adresse prüfen, Gateway pingen, öffentliche IP pingen, dann erst DNS testen. Mit ip a, ip route und ping lässt sich so fast jeder Fehler in wenigen Minuten eingrenzen.

Ich habe außerdem verstanden, wie die Windows Internetverbindungsfreigabe intern arbeitet, nämlich als kleiner Router mit eigenem DHCP und NAT, und warum Geräte dahinter aus dem Heimnetz nicht erreichbar sind. Bei Docker habe ich gesehen, wie Bridge Netzwerke und Portbindungen funktionieren und warum der Pi-hole die Tailscale Clients nur als Adresse der Docker Bridge sieht.

Und ich habe gelernt, YAML ernst zu nehmen. Einrückung ist dort Syntax, und eine Konfiguration als Ganzes zu ersetzen ist zuverlässiger als einzelne Zeilen zu flicken.